

Laporan Tugas Besar 1
IF3270 PEMBELAJARAN MESIN
FEED-FORWARD NEURAL NETWORK
SEMESTER II TAHUN 2025/2026



Muhammad Izzat Jundy	13523092
Zulfaqqar Nayaka Athadiansyah	13523094
Muhammad Adha	13523098

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

1	Deskripsi Persoalan	2
2	Analisis dan Pembahasan	2
2.1	Implementasi	2
2.2	Penjelasan Forward Propagation	4
2.3	Penjelasan Backward Propagation dan Weight Update	4
2.4	Hasil Pengujian	5
3	Kesimpulan dan Saran	7
4	Pembagian Tugas	7
	Referensi	7
A	Formulir Penggunaan AI	7

1 Deskripsi Persoalan

Tugas besar ini mengharuskan kami untuk mengimplementasikan *Feedforward Neural Network* (FFNN) dari nol (*from scratch*) menggunakan bahasa Python. Implementasi hanya diperbolehkan menggunakan library matematika seperti NumPy serta tidak diperkenankan menggunakan library deep-learning seperti PyTorch atau TensorFlow.

FFNN merupakan jenis jaringan syaraf tiruan di mana informasi hanya mengalir dari lapisan input ke lapisan output tanpa siklus/umpan balik. Model menerima vektor fitur sebagai input, memprosesnya melalui satu atau lebih lapisan tersembunyi (*hidden layer*), dan menghasilkan prediksi pada lapisan output.

Secara matematis, untuk satu lapisan ke lapisan berikutnya berlaku:

$$\mathbf{z}^{(l)} = \mathbf{a}^{(l-1)}\mathbf{W}^{(l)} + \mathbf{b}^{(l)} \quad \mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}) \quad (1)$$

di mana $\mathbf{W}^{(l)}$ adalah matriks bobot, $\mathbf{b}^{(l)}$ adalah vektor bias, dan $f^{(l)}$ adalah fungsi aktivasi pada lapisan ke- l .

Dataset yang digunakan untuk pengujian adalah *Global Student Placement & Salary Dataset*, dengan target prediksi berupa `placement_status` (status penempatan kerja). Makanya, persoalan yang di-handle di sini adalah klasifikasi.

2 Analisis dan Pembahasan

2.1 Implementasi

Implementasi FFNN ini dirancang secara modular untuk memisahkan logika struktur data, *layer* dari *neural network*, dan fungsi-fungsi pendukung lainnya. Arsitektur kode terdiri atas beberapa modul utama, mulai dari mesin *autograd* pada `tensor.py` hingga kelas pengelola model pada `ffnn.py`.

2.1.1 Kelas Tensor (`tensor.py`)

Tensor merupakan struktur data inti yang membungkus array NumPy untuk menyediakan kemampuan *automatic differentiation*. Di dalam implementasi kami, **Tensor** menyimpan nilai numerik utama pada atribut `data`, sementara akumulasi gradien hasil turunan disimpan pada atribut `grad`. Untuk membangun graf komputasi, setiap objek **Tensor** mencatat referensi ke "orang tua"-nya melalui atribut `_prev` dan menyimpan logika turunan spesifik operasi tersebut dalam fungsi anonim `_backward`. Metode yang paling krusial dalam kelas ini adalah `backward()`, yang melakukan *topological sort* pada graf komputasi untuk memastikan gradien dihitung dalam urutan yang benar dari belakang ke depan. Selain itu, kelas ini meng-overload operator aritmatika standar Python sehingga operasi seperti penjumlahan atau perkalian matriks secara otomatis mendaftarkan node baru dalam graf komputasi.

Table 1: Atribut dan Metode Kelas **Tensor**

Nama	Tipe	Deskripsi
<code>data</code>	<code>np.ndarray</code>	data numerik utama (<code>float64</code>)
<code>grad</code>	<code>np.ndarray</code>	akumulasi gradien turunan
<code>requires_grad</code>	<code>bool</code>	penanda apakah gradien perlu dihitung
<code>backward()</code>	<code>method</code>	memulai proses <i>backpropagation</i>
<code>zero_grad()</code>	<code>method</code>	me-reset gradien menjadi nol.

2.1.2 Layer (`layer.py`)

Lapisan **Linear** merepresentasikan satu lapisan *fully connected* yang melakukan transformasi linear terhadap data melalui operasi $\mathbf{z} = \mathbf{a}_{\text{prev}}\mathbf{W} + \mathbf{b}$. Atribut utama kelas ini adalah `W` dan `b` yang disimpan dalam objek **Tensor** untuk memungkinkan pelacakan gradien otomatis. Kelas ini mendukung berbagai metode inisialisasi bobot seperti *Zero*, *Uniform*, *Normal*, *Xavier*, hingga *He*.

Metode `forward(x)` bertanggung jawab untuk menghitung hasil transformasi linear dari input yang diberikan. Selain itu, tersedia metode `backward()` manual yang secara eksplisit menghitung gradien terhadap bobot, bias, dan input sebelumnya ($\partial C / \partial \mathbf{a}_{\text{prev}}$).

Table 2: Atribut dan Metode Kelas **Linear**

Nama	Tipe	Deskripsi
<code>W</code>	Tensor	matriks bobot berukuran (in × out)
<code>b</code>	Tensor	vektor bias berukuran (out,)
<code>forward(x)</code>	method	menghitung transformasi linear $x\mathbf{W} + \mathbf{b}$
<code>backward()</code>	method	menghitung gradien analitik untuk bobot dan bias
<code>parameters()</code>	method	mengembalikan daftar parameter <code>[W, b]</code>

2.1.3 Fungsi Aktivasi (`activation.py`)

Setiap kelas aktivasi mengimplementasikan `__call__(x: Tensor)`. Karena menggunakan **Tensor**, `backward` secara otomatis ditangani oleh *autograd*.

Table 3: Fungsi aktivasi dan turunannya

Nama	Definisi $f(z)$	Turunan $f'(z)$
Linear	z	1
ReLU	$\max(0, z)$	$\mathbf{1}[z > 0]$
Sigmoid	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$\sigma(z)(1 - \sigma(z))$
Tanh	$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2(z)$
Softmax	$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$	(matriks Jacobian; dikombinasikan dengan CCE)

Di sini implementasi Softmax kami pakai *numerical stability trick* $z' = z - \max(z)$ sebelum eksponen dihitung. Ini biar tidak terjadi *overflow* saat eksponen dihitung.

2.1.4 Fungsi Loss (`loss.py`)

Table 4: Fungsi loss yang diimplementasikan

Nama	Definisi
MSE	$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$
Binary Cross-Entropy	$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)]$
Categorical Cross-Entropy	$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^C y_{ij} \ln \hat{y}_{ij}$

Kami pakai `Tensor.clip` untuk menghindari $\ln(0)$.

2.1.5 Kelas **BackwardRules** (`backward.py`)

Kelas **BackwardRules** bertindak sebagai *library* turunan analitik yang dipakai ketika model dijalankan dalam mode "manual". Di sini `dC_da()` menghitung turunan *loss* terhadap output lapisan terakhir, sementara `da_dz()` menghitung turunan fungsi aktivasi terhadap pre-aktivasi.

Table 5: Metode pada Kelas **BackwardRules**

Nama	Tipe	Deskripsi
<code>dC_da()</code>	static	turunan fungsi <i>loss</i> terhadap prediksi ($\partial \mathcal{L} / \partial \hat{y}$)
<code>da_dz()</code>	static	turunan fungsi aktivasi terhadap pre-aktivasi ($\partial a / \partial z$)

2.1.6 Kelas FFNN (ffnn.py)

FFNN merupakan kelas utama yang menggabungkan seluruh komponen lapisan, aktivasi, dan fungsi *loss* menjadi satu model utuh. Kita dapat menentukan arsitektur jaringan melalui parameter `layer_sizes`, metode inisialisasi, hingga teknik regularisasi L1 atau L2. Kelas ini memiliki mekanisme `_cache` internal untuk menyimpan data antara selama *forward pass* yang dibutuhkan dalam proses *backpropagation* manual.

Metode `fit()` mengelola siklus *trainig* menggunakan *mini-batch Stochastic Gradient Descent* (SGD).

Table 6: Atribut dan Metode Kelas FFNN

Nama	Tipe	Deskripsi
<code>layers</code>	<code>list</code>	daftar objek <code>Linear</code> penyusun jaringan
<code>_cache</code>	<code>dict</code>	penyimpanan sementara input dan pre-aktivasi
<code>forward(x)</code>	<code>method</code>	melakukan aliran data maju dari input ke output
<code>fit()</code>	<code>method</code>	menjalankan loop pelatihan dengan mini-batch SGD
<code>predict()</code>	<code>method</code>	melakukan inferensi kelas atau probabilitas
<code>save/load()</code>	<code>method</code>	melakukan serialisasi parameter model ke file

2.2 Penjelasan Forward Propagation

Forward propagation mengalirkan data dari lapisan input ke output secara berurutan. Method `forward(x)` pada FFNN melakukan langkah berikut:

1. Input $\mathbf{X} \in \mathbb{R}^{n \times d}$ (batch n sampel, d fitur) dibungkus sebagai `Tensor`.
2. Untuk setiap pasangan $(\text{layer}_l, \text{activation}_l)$:
 - (a) Simpan $\mathbf{a}^{(l-1)}$ ke `_cache["inputs"]`.
 - (b) Hitung pre-aktivasi: $\mathbf{z}^{(l)} = \mathbf{a}^{(l-1)}\mathbf{W}^{(l)} + \mathbf{b}^{(l)}$.
 - (c) Simpan $\mathbf{z}^{(l)}$ ke `_cache["pre_acts"]`.
 - (d) Hitung aktivasi: $\mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)})$.
3. Kembalikan $\mathbf{a}^{(L)}$ (output lapisan terakhir).

Karena semua operasi dilakukan dengan `Tensor`, graf komputasi (DAG) otomatis terbangun selama forward pass dan siap digunakan oleh `backward()`.

2.3 Penjelasan Backward Propagation dan Weight Update

2.3.1 Mode Autograd

Pada mode "autograd", setelah loss dihitung, cukup dipanggil `loss.backward()`. Metode ini:

1. Melakukan *topological sort* pada DAG komputasi.
2. Menginisialisasi $\partial \mathcal{L} / \partial \mathcal{L} = 1$.
3. Memanggil `_backward()` setiap node dalam urutan terbalik (*reverse mode*), mengakumulasi gradien ke `.grad` tiap `Tensor`.

Dengan pendekatan ini, setiap operator (penjumlahan, perkalian matriks, exp, dll.) bertanggung jawab mendefinisikan turunannya sendiri melalui *closure* dalam `_backward`.

2.3.2 Mode Manual (Chain Rule Eksplisit)

Pada mode "manual", `_manual_backward` menggunakan aturan dari `BackwardRules` secara eksplisit:

1. Hitung $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}}$ menggunakan `BackwardRules.dC.da`.
2. Untuk $l = L, L-1, \dots, 1$ (dari output ke input):
 - (a) Hitung $\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}} \odot \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}}$ menggunakan `BackwardRules.da.dz`.
 - (b) Hitung gradien bobot: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = (\mathbf{a}^{(l-1)})^\top \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$
 - (c) Hitung gradien bias: $\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \sum_{\text{batch}} \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}}$
 - (d) Propagasikan ke lapisan sebelumnya: $\frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l-1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{(l)}} (\mathbf{W}^{(l)})^\top$
3. Akumulasikan gradien regularisasi (L1/L2) ke `W.grad`.

2.3.3 Regularisasi

Term regularisasi ditambahkan ke loss sebelum backward:

$$\text{L1 : } \mathcal{L}_{\text{reg}} = \lambda \sum_l \|\mathbf{W}^{(l)}\|_1 \quad \Rightarrow \quad \nabla_{\mathbf{W}} \mathcal{L}_{\text{reg}} = \lambda \text{sign}(\mathbf{W}) \quad (2)$$

$$\text{L2 : } \mathcal{L}_{\text{reg}} = \lambda \sum_l \|\mathbf{W}^{(l)}\|_2^2 \quad \Rightarrow \quad \nabla_{\mathbf{W}} \mathcal{L}_{\text{reg}} = 2\lambda \mathbf{W} \quad (3)$$

2.3.4 Weight Update (Gradient Descent)

Setelah gradien dihitung, bobot diperbarui dengan aturan:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} \quad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} \quad (4)$$

di mana η adalah *learning rate*.

2.4 Hasil Pengujian

Bagian ini menyajikan hasil eksperimen model FFNN yang dijalankan melalui Jupyter Notebook. Pengujian mencakup variasi arsitektur, parameter pelatihan, hingga perbandingan performa dengan pustaka standar Scikit-learn untuk memvalidasi implementasi kami.

2.4.1 Pengaruh Width dan Depth

Eksperimen terhadap lebar jaringan (*width*) menunjukkan bahwa model yang lebih kompleks cenderung mengalami *overfitting* pada dataset ini. Penggunaan `Width-16` memberikan hasil yang paling seimbang dengan akurasi tes sebesar 0.7500. Sebaliknya, pada `Width-256`, akurasi latih mencapai angka sangat tinggi (0.8671) namun akurasi tes turun menjadi 0.6900, yang menandakan model terlalu menghafal data latih.

Sementara itu, pengujian kedalaman (*depth*) mengindikasikan bahwa struktur yang lebih dangkal jauh lebih efektif. Akurasi tes tertinggi didapatkan pada `Depth-1` sebesar 0.7433, dan performa menurun drastis hingga 0.6153 saat kedalaman ditingkatkan menjadi `Depth-5`. Hal ini menunjukkan bahwa penambahan lapisan tersembunyi yang terlalu banyak justru mempersulit proses konvergensi untuk kasus ini.

Table 7: Ringkasan Pengaruh Variasi Arsitektur

Konfigurasi	Train Acc	Val Acc	Test Acc
Width-16	0.7601	0.7480	0.7500
Width-64	0.7506	0.7180	0.7480
Width-256	0.8671	0.6880	0.6900
Depth-1	0.7591	0.7413	0.7433
Depth-3	0.7043	0.6780	0.6833
Depth-5	0.6153	0.6153	0.6153

2.4.2 Pengaruh Fungsi Aktivasi

Pengujian terhadap berbagai fungsi aktivasi pada lapisan tersembunyi menunjukkan hasil yang cukup kompetitif. Menariknya, fungsi **sigmoid** dan **linear** memberikan akurasi tes yang sedikit lebih tinggi (0.748) dibandingkan dengan **relu** (0.7260). Hal ini menunjukkan bahwa untuk dataset spesifik ini, transisi nilai yang lebih halus atau representasi linear yang sederhana pada lapisan awal memberikan kontribusi positif terhadap prediksi status penempatan kerja.

Table 8: Hasil Variasi Fungsi Aktivasi

Aktivasi	Train Acc	Val Acc	Test Acc
Linear	0.7450	0.7360	0.7473
ReLU	0.7359	0.7213	0.7260
Sigmoid	0.7593	0.7513	0.7487
Tanh	0.7561	0.7333	0.7433

2.4.3 Pengaruh Learning Rate

Eksperimen terhadap *learning rate* (LR) menunjukkan pentingnya pemilihan nilai yang moderat. Nilai $lr=0.01$ memberikan stabilitas yang baik selama proses pelatihan. Meskipun peningkatan LR ke 0.1 meningkatkan akurasi latih menjadi 0.7937, akurasi tes yang dihasilkan tetap sama dengan konfigurasi LR yang lebih kecil, yang menandakan bahwa peningkatan kecepatan belajar tidak selalu menghasilkan model yang lebih cerdas dalam menggeneralisasi data baru.

Table 9: Hasil Variasi Learning Rate

Learning Rate	Train Acc	Val Acc	Test Acc
$lr = 0.001$	0.7130	0.6887	0.7187
$lr = 0.01$	0.7359	0.7213	0.7260
$lr = 0.1$	0.7937	0.7187	0.7260

2.4.4 Pengaruh Regularisasi

Penerapan regularisasi L1 dan L2 terbukti efektif dalam meningkatkan performa model. Model yang menggunakan regularisasi L1 berhasil mencapai akurasi tes 0.7500, lebih tinggi dibandingkan model standar (**normal**) yang berada di angka 0.7260. Hal ini menunjukkan bahwa teknik *weight decay* atau penalti pada bobot membantu model untuk mengurangi kompleksitas yang tidak perlu dan fokus pada fitur-fitur yang paling relevan bagi prediksi.

Table 10: Hasil Variasi Regularisasi

Konfigurasi	Train Acc	Val Acc	Test Acc
Normal (No Reg)	0.7359	0.7213	0.7260
L1 ($\lambda = 1e-3$)	0.7520	0.7340	0.7500
L2 ($\lambda = 1e-3$)	0.7543	0.7413	0.7473

2.4.5 Perbandingan dengan *Scikit-Learn*

Sebagai bentuk validasi akhir, model FFNN *custom* kami diuji secara langsung melawan **MLPClassifier** dari Scikit-Learn menggunakan parameter yang setara. Hasilnya menunjukkan bahwa implementasi manual kami berhasil mengungguli Scikit-Learn dengan akurasi tes 0.7480 berbanding 0.7120. Keunggulan

ini memberikan bukti kuat bahwa mesin *autograd* dan algoritma propagasi balik yang kami bangun dari nol telah berjalan secara akurat dan sangat kompetitif.

Table 11: Perbandingan Metrik FFNN Custom vs Scikit-Learn MLP

Metrik (Weighted Avg)	FFNN Custom	Scikit-Learn MLP
Akurasi	0,7480	0,7120
Precision	0,76	0,72
Recall	0,75	0,71
F1-Score	0,73	0,71

3 Kesimpulan dan Saran

Implementasi model *Feedforward Neural Network* (FFNN) dari nol berhasil membuktikan akurasi dan validitas algoritma propagasi balik dengan melampaui performa model pembandingan Scikit-Learn pada akurasi tes sebesar 0,7480 berbanding 0,7120. Melalui serangkaian eksperimen, ditemukan bahwa arsitektur yang lebih dangkal dan ramping, seperti konfigurasi satu lapisan tersembunyi dengan 16 neuron, memberikan hasil optimal dalam mencegah *overfitting* pada dataset status penempatan kerja. Selain itu, penggunaan fungsi aktivasi sigmoid serta penerapan teknik regularisasi L1 terbukti berperan penting dalam meningkatkan kemampuan generalisasi model, sehingga menghasilkan prediksi yang lebih stabil dan tangguh dibandingkan konfigurasi arsitektur yang lebih kompleks.

4 Pembagian Tugas

Table 12: Pembagian tugas anggota kelompok

NIM	Nama	Tugas
13523092	Muhammad Izzat Jundy	
13523094	Zulfaqqar Nayaka Athadiansyah	
13523098	Muhammad Adha Ridwan	

Referensi

- Karpathy, A. *The Spelled-Out Intro to Neural Networks and Backpropagation: Building Micrograd*. <https://youtu.be/VMj-3Sitku0>
- Osajima, J. *Forward Propagation Explained*. <https://www.jasonosajima.com/forwardprop>
- Osajima, J. *Backward Propagation Explained*. <https://www.jasonosajima.com/backprop>
- NumPy Documentation. <https://numpy.org/doc/2.2/>
- scikit-learn. *MLPClassifier*. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
- OpenStax Calculus. *The Chain Rule for Multivariable Functions*. [https://math.libretexts.org/Bookshelves/Calculus/Calculus_\(OpenStax\)/14%3ADifferentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariate_Functions](https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3ADifferentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariate_Functions)
- Green, E. *The Softmax Function and Its Derivative*. <https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/>
- Orr, D. *Building Autodiff from Scratch*. <https://douglasorr.github.io/2021-11-autodiff/article.html>
- Grosse, R. *CSC2541 Tutorial: Automatic Differentiation*. https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2022/tutorials/tut01.pdf
- Built In. *L2 Regularization*. <https://builtin.com/data-science/l2-regularization>

A Formulir Penggunaan AI